
STAT 184 Final Report: Q-Learning-Based Sensor Tasking for Enhanced Space Domain Awareness

Theodore McCulloch
Harvard University
Cambridge, MA 02138
tmcculloch@g.harvard.edu

Chirag Mahapatra
Harvard University
Cambridge, MA 02138
chiragmahapatra@hks.harvard.edu

Abstract

This project applies Q-Learning techniques, including Tabular Q-Learning and Deep Q-Networks (DQN), to optimize sensor tasking decisions for monitoring maneuverable space objects. Efficient sensor allocation is crucial for tracking satellites and space debris, but traditional methods often struggle with the dynamic and complicated nature of space environments. Therefore, we utilize Q-Learning to automate decision-making for sensor tasking. The results demonstrate the potential of reinforcement learning to improve space domain awareness (SDA) and sensor management in real-time operations.

1 Introduction

The Space Surveillance Network (SSN) (Figure 1) is a system of ground and space based sensors, RADAR stations, telescopes, and communication sites [1] that monitor objects (debris, rocket bodies, active/inactive satellites, launches, etc.) within the Earth's orbit for the purpose of space domain awareness (SDA) [2]. There are multiple actors, typically United States agencies, with tasking access to the SSN—each with their own set of concerns. Consolidated tasking orders (lists of satellites, assigned to sensors) are published (secretly, not publicly) in order to maintain a baseline-level of situational awareness. However, often individual actors will find that either the cadence, quality, or geometry of these observations and resulting states are insufficient for their individual operational needs. For example, imagine an adversarial object with high maneuverability; if this object can escape custody within minutes, it is insufficient to observe it with the cadence of hours.

Moreover, the space domain is a crowded and contested environment with complexity only increasing (Figure 2). The inevitable rise of refuellable on-orbit satellites will further complicate matters, introducing objects that can conduct large, strategic maneuvers without the current, high-level of consideration for fuel constraints. This new era has been coined “The Third Age of Space” and will necessitate rethinking longstanding sensor operation assumptions in support of Dynamic Space Operations (DSO) [3].

Solving the optimization problem of the SSN is beyond the scope of this project. Rather, we are seeking to understand how well RL methodology can function as a decision-maker in the SDA space. Part of this is exploring what information from the environment we should encode (and how) in order to incentivize maintaining custody of variable objects with variable sensors without spamming the SSN. Reinforcement learning offers a promising avenue for addressing these challenges by enabling adaptive, real-time decision-making that can optimize sensor allocation and maintain custody of key space objects. For this project, we are utilizing both Tabular Q-Learning, Linear Q-Function Approximation, and Deep Q-Networks (DQN) to optimize sensor tasking decisions. Tabular Q-learning is used for simpler, discrete environments where a table of state-action values can be easily maintained, while the function approximation approach—seen in Linear and DQN—is able to scale in order to handle more complex, high-dimensional state spaces. Together, these methods offer a flexible approach to learning optimal tasking strategies in both simpler and more challenging scenarios.

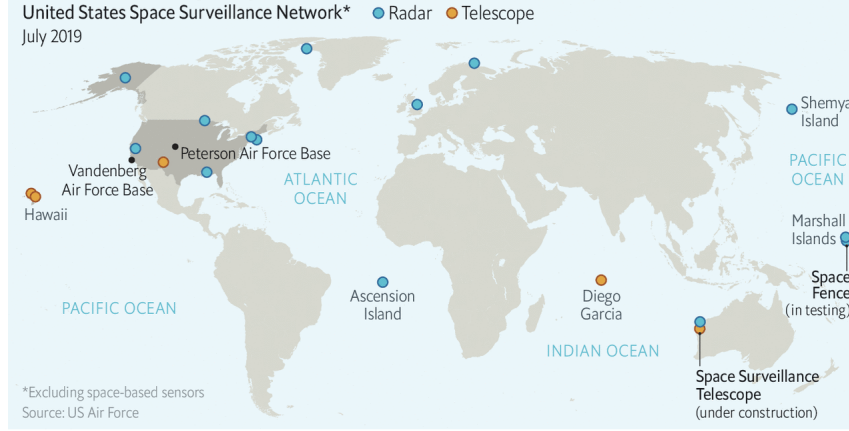


Figure 1: **United States Space Surveillance Network (SSN)**
Map of fixed, ground RADARs and telescopes within the SSN [8].

2 Environment

Our environment includes sensors, satellites, their statuses, and information flowing between agents and sensors. In order to limit problem complexity, we have made two key choices to scope our environment. The first is, we will only consider ground RADARs and telescopes when modeling the SSN. In reality, there are a series of space-based sensors which also make contributions to the state catalog; however, these come with a series of modeling complexities which include up-link/downlink timing calculations, object-specific visibility models, and unique data latency patterns. Since their contribution is less important than ground sensors, they can be reasonably excluded from the environment.

The second choice is, we are restricting our satellites to geosynchronous and geostationary orbital regimes (Figure 3). Low Earth Orbit (LEO) is the most crowded regime. Including orbits ranging from 160 [km] to 2,000 [km] above the Earth’s surface, it contains about 55 [%] of all currently operational satellites [4]. Notable examples include the International Space Station (ISS) [5] and Starlink constellation [6]. Although this regime is the most crowded, it is also the most observed, as it contains a high number of commercial assets. The GEO regime contains 35 [%] of operational satellites [4], but these tend to be more expensive, government-owned objects. For this reason, when rethinking SSN sensor operations, these objects are more relevant than LEO satellites. Moreover, GEO satellites have the nice property of orbiting at roughly the same rate of the Earth [7], meaning that if they are within the line of sight of a sensor, they typically will remain so (unless a large maneuver is conducted).

2.1 Data Source

Satellite states are frequently published to space-track.org in order to promote flight safety and to assist astronomers [10]. Unfortunately, maneuver and observation data are either secret or proprietary. Satellite information is often encoded in TLE (Two Line Element) form [11], which is a legacy format that contains Keplerian elements detailing the geometry of a satellite’s orbit. However, these can be converted into satellite state vector (x, y, z, dx, dy, dz) form using Skyfield [12] an opensource astrodynamics, Python library. We are also using Skyfield to conduct propagation, sensor-satellite angles for visibility, and solar positioning. Impulsive maneuvers are simulated by making small perturbations to the velocity vector and then re-estimating the orbital elements based on the change.

2.2 Key Components

Our environment contains several important components, which are detailed below. Exactly how they all interact is summarized by Algorithm 1.

Satellite Class - This class keeps track of the truth values for a given satellite. This truth is initialized via TLE but is re-epochd relative to the scenario start time in order to add control when specifying

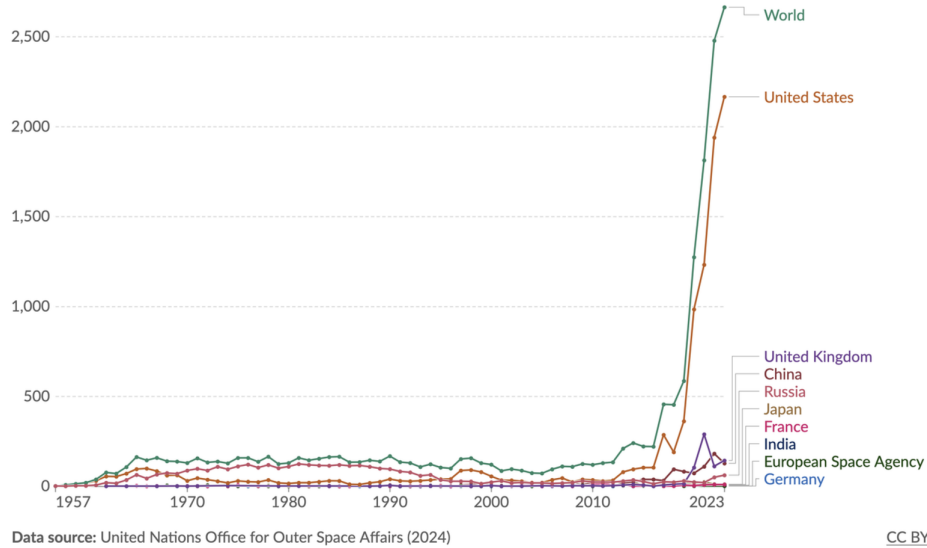


Figure 2: **Growth in Annual Launches of Objects into Space**

This figure shows the growth in annual launches since the launch of Sputnik 1 in 1957; it demonstrates that the United States makes up the majority of these launches (roughly 81% of the global total in 2023) [9].

the scenario as well as to limit propagation time. If a satellite maneuvers during the scenario, this information is also stored here. A given satellite is able to maneuver multiple times during a scenario or not at all.

Maneuver Class - This class stores the magnitude, direction, and timing of a given maneuver. This information is provided to the Satellite class, which is responsible for re-estimating the truth state based on this information. The number, timing, and/or magnitude of maneuvers are randomized during training.

Sensor Class - This class handles visibility calculations, receiving of task messages (with randomized delays), the scheduling of tasks, and the issuing of different event notifications (also with randomized delays). All sensors are ground sites and can either be RADARs or optical telescopes. Scheduling logic is first-come-first-serve, just-in-time, and there is a specified time limit on how far a sensor can schedule tasking-time in advance.

State Catalog This object keeps track of the known states of each satellite, and when they were last observed. If there is more than one agent in the scenario, they all share this catalog, meaning agents can benefit from each other's tasking if they share assigned satellites.

Agent Wrapper This object receives the raw state from the environment and issues actions based on its internal algorithm. It encodes the current, knowable environment's state into a usable form for the RL agent as well as decodes the agent's actions into usable information for the environment.

2.3 Events

Our environment provides notifications about several event types, which contain useful information about the results of a given agent's tasking requests and the information gained, therein.

1. Invalid Tasking Request - type 1 - this happens when an agent issues tasking to a sensor that doesn't have visibility on the satellite. We are working with geocentric/geosynchronous satellites, which means that if their states are nominal (i.e. have not maneuvered), they will remain visible to the same sensors. We chose not to add predicted visibility to the state, as it would dramatically increase the state size. Instead we issue a penalty resulting from this event, to dis-incentivize repeating this action.

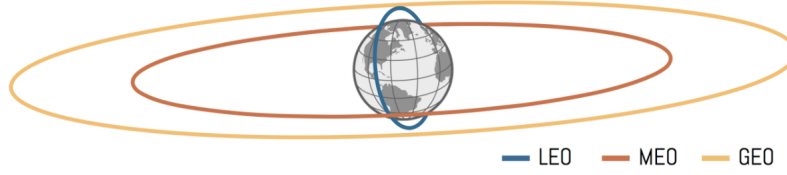


Figure 3: **Orbital Regimes**

There are three common orbital regimes: low Earth orbit (LEO), medium Earth orbit (MEO), and geosynchronous/geostationary Earth orbit (GEO) [4].

2. **Invalid Tasking Request - type 2** - This event occurs when the sensor does have line of sight but is not operational due to time of day constraints. Since we assume RADARs in the SSN are always operational, it can only occur when tasking optical sensors, which only function during the night. We are defining "night" as the sun being at least 12 [degrees] below the horizon at the sensor site. This delineation is known as "astronomical twilight" [13], and it is standard practice for optical telescopes to adhere to these hours.

3. **Dropped Task** - this event notification is issued by the sensor if it was unable to schedule a tasking request due to tasking conflicts. A small penalty is issued for this event for the purpose of discouraging spamming the SSN.

4. **Lost Object** - this event is issued by the sensor if an object maneuvered out of its field of view. This doesn't necessarily mean that the satellite is not still theoretically visible to the sensor; rather it means the sensor would have to conduct searching in order to recover the satellite. We decided to not implement a recovery mechanism in our environment, so once a satellite is lost the agent will continue to be punished for its loss. We made this choice as loss of custody is in practice a failure state (i.e. not an option) for High Interest Objects (HIOs).

5. **Completed - Nominal** - this event occurs if the sensor has successfully completed tasking on an object, discovering that the satellite state was nominal (i.e.: not maneuvered). These notifications are only provided to the agent that issued the associated task message. They occur when the updated state reaches the catalog.

6. **Completed - Maneuver** - this event occurs if the sensor has successfully completed tasking on an object, discovering satellite has maneuvered (but custody was not lost). The agent that issued the associated task message will be rewarded. Only one reward is issued per maneuver, which is granted to the agent whose tasking manages to update the state catalog first.

3 Training Specifics

3.1 State Encoding

We experimented with several ways of encoding the state. Initially, we wanted to include information about incoming events encoded as integers. However, after experimenting with different state encodings, we found that the addition of events in this manner had no perceivable impact on the training outcome. Instead, our state contains two pieces of information about each satellite: the last time it was seen (state catalog age) and the last time the agent sent out a tasking message. These are discretized into ranges for the Q-Table but left as float values for the function approximation methods. Therefore, if we have n satellites, our state can be represented by an $2n \times 1$ array.

3.2 Action Encoding

At each timestep an agent is able to issue tasking on all of or none of the satellites. The sole limitation is each satellite can only be tasked once per timestep, in order to limit the action space complexity. Therefore, if we have n satellites and m sensors, our tasking action can be encoded as an $n \times 1$ array of values, where each position in the array can take on $m + 1$ different values—specifically an integer within the range of $[-1, m - 1]$. Where values > -1 represent the index of a sensor and -1 represents taking no action. This results in an action space of size $|A| = (m + 1)^n$.

Algorithm 1 summary of the environment algorithm run during a given scenario/trajectory

Require: Satellite(s) (O), Sensor(s) (S), Agent(s) (A), Scenario Specifics (I)

```
truthsat ← initTruth( $S, I$ )           ▷ initialize ground truth; estimate maneuvers
 $t$  ←  $I.scenarioEpoch$                  ▷ initialize start time
 $C$  ← initCatalog( $S, I$ )               ▷ initialize state catalog
while  $t < I.scenarioEndTime$  do
  truthsat.advance( $t$ )
  events ← []                         ▷ check for available events
  for sensor in  $S$  do
    events.add(sensor.events( $t$ ))     ▷ advance truth in time
  end for
   $C.update(events, t)$                 ▷ update catalog if necessary
  actions ← []                        ▷ gather actions (task requests) from agents
  for agent in  $A$  do
    actions.add(agent.decide( $t$ ))
  end for
  for task in actions do              ▷ send tasks to appropriate sensors
    for sensor in  $S$  do
      if task.involves(sensor) then
        sensor.receiveWithDelay(task,  $t$ )  ▷ sensors won't receive tasks instantly
      end if
    end for
  end for
  for sensor in  $S$  do                  ▷ advance sensors in time
    sensors.advance( $t$ )
  end for
   $t$  +=  $I.dt$                           ▷ advance time
end while
```

3.3 Cost-Reward Function

Reward shaping was one of our primary strategies to coax our agents into learning useful policies. This being the case, we have gone through many different iterations of the reward function with numerous weighting schemes. The considerations and the final testing weighting scheme are outlined in Table 1.

Maneuver Detected	Successful Task	Dropped (scheduling)	Dropped (Lost)
+10	+2	-3	-1000
Invalid Task (type 1)	Invalid Task (type 2)	Last Seen > 90 [mins]*	
-50	-2	-1	

Table 1: Summary of rewards and penalties issued for events.

*penalty is issued at each timestep per object for which condition is true.

3.4 Scenarios

We designed a handful of scenarios with increasing levels of complexity to train and test our agents (see Table 2); each scenario has a training counterpart, which is simplified version of the deployed, testing environment. In this way, we can determine how well an agent’s policy generalizes outside of the training environment. Both the training and test environments always randomize: (1) the scenario start time, (2) the starting state catalog age of object, (3) the maneuver times of objects, (4) the maneuver sizes of objects, and (5) (occasionally) the length of scenario. The testing environments additionally randomize (6) the number of maneuvers per object. All randomization occurs at the beginning of each episode.

	Objects	RADAR(s)	Telescope(s)	Man.(s) [per obj.]	Length [hours]
<i>S1 - Train</i>	2	1	0	1	6
<i>S1 - Test</i>	2	1	0	2-4	48
<i>S2 - Train</i>	8	2	1	1-4	6-12
<i>S2 - Test</i>	8	2	1	0-5	32-48

Table 2: Quick high-level comparison and summaries of the different scenarios, including the variations between the training and testing environments.

4 RL Methodology

4.1 Tabular Q-Learning

Tabular Q-learning is a model-free, off-policy algorithm that updates a Q-Table (state-action value function) to determine the best action to take in each state. Since the SSN is a dynamic environment with shifting resource availabilities and emerging events, we hypothesized that Q-Learning would be well-suited as an optimization strategy, as it can adapt over time based on feedback from the environment, without necessitating a detailed or explicit model.

The core logic of Tabular Q-Learning is to iteratively update $Q(s, a)$ based on the reward observed and the estimated future rewards from the next state:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

The primary hyper-parameters are (1) the learning rate ($\alpha \in [0, 1]$) and (2) the discount factor ($\gamma \in [0, 1]$), which control the rate new information overrides old information and the importance of future rewards, respectively. [14]

4.2 Linear Q-Function

The major drawback to Tabular Q-Learning is its limited scalability. To get around this, we turned to function approximation. Q-Learning with function approximation is an extension of traditional Q-Learning where instead of maintaining a Q-Table for each state-action pair, a function is used to approximate the Q-values for large or continuous state spaces. This allows Q-Learning to scale to more complex environments by generalizing across similar states rather than storing values for every possible state-action pair.

In the case of linear function approximation, the Q-value for a state-action pair is approximated as a linear combination of features derived from the state-action pair. This changes our update rule to following:

$$\theta \leftarrow \theta + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a; \theta) - Q(s_t, a_t; \theta)] \nabla_{\theta} Q(s_t, a_t; \theta)$$

Where,

- $Q(s, a; \theta) = \theta^T \phi(s, a)$: estimated Q-value for state s and action a
- θ : vector of learned weights
- $\phi(s, a)$: feature vector corresponding to state s and action a
- $\nabla_{\theta} Q(s_t, a_t; \theta) = \phi(s_t, a_t)$: gradient of the Q-value function with respect to θ

[15]

4.3 DQN: Neural Network to Approximate Q-Function

The DQN agent utilizes a neural network to approximate the Q-function, hypothetically enabling it to handle the high-dimensional state-action space of sensor tasking efficiently. The agent's network consists of three fully connected layers, with ReLU activation functions in the first two layers and a final layer outputting action values scaled by a hyperbolic tangent activation. The agent employs experience replay to store transitions in a replay buffer, sampling batches during training to decorrelate updates. For each transition, the Q-value target is computed as $q_{\text{target}} = r + \gamma \max_a Q_{\text{target}}(s', a)$, and the mean squared error is minimized using the Adam optimizer. The policy is trained with an

epsilon-greedy exploration strategy, where epsilon decays exponentially to encourage exploration early in training and exploitation of learned policies later. The model parameters are updated using gradient clipping to stabilize training and avoid exploding gradients.

5 Experiment Results

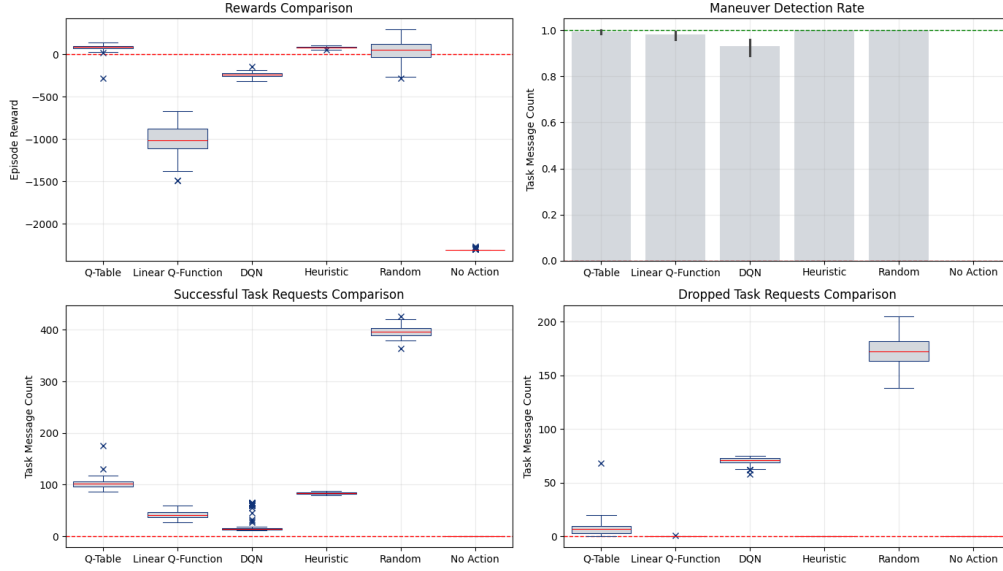


Figure 4: **Scenario 1 - Test Results: Algorithm Generalization Performance**

These plots compare the performance of different algorithms within the Scenario 1 Environment, using total rewards, maneuver detection rate, successful task requests, and dropped task requests as metrics for success/failure.

5.1 Tabular Q-Learning

Overall our Q-Table performance was relatively strong. Looking at Figures 4 and 5, we see that in terms of rewards, its performance did not vary much between test experiments, indicating that it developed a stable policy that generalized well to the testing environment. In Scenario 1, it performed about as well as the heuristic tasking algorithm in terms of rewards. Moreover, it had a 100 [%] maneuver detection rate for all Scenario 1 test runs. Its maneuver detection rate suffered in Scenario 2, but, otherwise, performance remained consistently similar to the heuristic algorithm's.

The reason that Scenario 2 doesn't have more objects, is because we were unable to scale beyond eight given our encoding of the state and implementation of the Q-Table. This highlights the primary drawback of Tabular Q-Learning: while straightforward to implement and relatively easy to train, it does not scale well.

5.2 Linear Function Approximation

We had less success than with the Q-Table approach when turning to Linear function approximation. Initially, we were only going to include Q-Table and DQN in our project; however, as we will discuss in the next section, we were not able to get DQN to settle into a stable policy. For this reason, we wanted to add another function approximation technique for comparison.

For Scenario 1, Linear Q-Learning performed worse than the Q-Table and DQN approaches. The majority of the penalties it incurred were from letting object epoch ages surpass 90 [mins] and then issuing tasking. Therefore, its schedules resemble the Q-Table's but the Q-Table approach was able to avoid these penalties by issuing tasking preemptively. It was less successful in Scenario 2. The primary issue was policy stability, as we found that different runs often yielded very different behavior. This accounts for the large ranges seen in Figure 5.

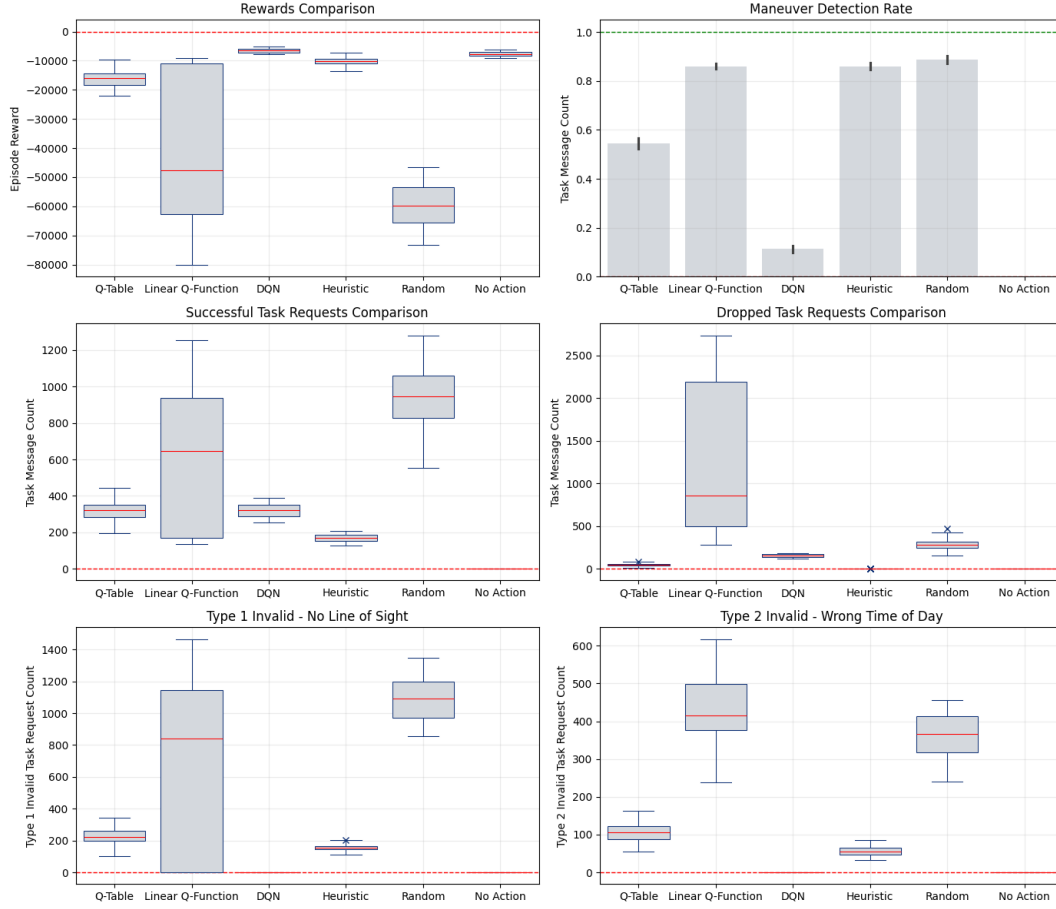


Figure 5: **Scenario 2 - Test Results: Algorithm Generalization Performance Results**

These plots compare the performance of different algorithms within the Scenario 2 test environment, using total rewards, maneuver detection rate, successful task requests, and dropped/invalid task requests as metrics for success/failure.

5.3 DQN

The majority of debugging, training, and testing time and effort went into trying to coax stable behavior from DQN. The primary issue we saw, was that neural network tended learn a "do (almost) nothing" policy strategy. We attribute this to the size of the continuous state space, which prevented our epsilon-greedy exploration from learning a useful policy. To combat this, we allowed the epsilon-greedy exploration to not be random but rather follow the strategy of the heuristic algorithm. For smaller scenarios, this introduced better policy behavior. In fact, DQN did outperform its Linear counterpart in Scenario 1 in all categories except maneuver detection rate (see Figure 4). This did not scale to larger scenarios, however. When looking at the Scenario 2 results (see Figure 5), it may initially appear that DQN is performing fairly well. However, this is because it has adopted a do next to nothing policy, making it marginally more useful than taking no action at all.

Given more time, we would implement proper Imitation Learning into our DQN training pipeline, since this appeared to show some promise in smaller experiments. We could also use cross-validation when tuning our hyper-parameters. Unfortunately, because the complexity of the class and algorithm, each run of DQN is far more time and computationally expensive than our other algorithms. This makes experimenting with tuning very time-consuming.

6 Conclusion

The application of Q-Learning techniques, including Tabular Q-Learning, Linear Function Approximation, and Deep Q-Networks, in this project highlights the potential of reinforcement learning to optimize sensor tasking for space domain awareness (SDA). While Tabular Q-Learning demonstrated stability and strong performance in simpler scenarios, its limitations in scalability necessitated exploring other approaches. The function approximation methods showed mixed results, with Linear Q-Function struggling with policy stability and DQN encountering challenges in scaling to more complex environments. However, these experiments provided valuable insights into the strengths and weaknesses of these methodologies.

Looking ahead, refining the reward structures, leveraging imitation learning, and optimizing hyperparameters could enhance performance, particularly for DQN models in high-dimensional state spaces. By advancing these techniques, reinforcement learning can play a critical role in addressing the growing complexity of space operations and enabling real-time, adaptive decision-making for SDA. This project lays the groundwork for further exploration and innovation in this crucial area.

References

- [1] U.S. Space Command. *Image: U.S. Space Command Imagery*. Accessed: December 9, 2024. 2023. URL: <https://www.spacecom.mil/MEDIA/IMAGERY/igphoto/2002163590/>.
- [2] National Security Archive. *What's There, Where Is It, and What's It Doing? The U.S. Space Surveillance System*. Accessed: December 9, 2024. 2023. URL: <https://nsarchive.gwu.edu/briefing-book/intelligence/2023-03-13/whats-there-where-it-and-whats-it-doing-us-space-surveillance>.
- [3] John E. Shaw, David R. Bourque, and Susan Shaw. "Dynamic Space Operations: The New Sustained Space Maneuver Imperative". Draft. 2023.
- [4] Center for Strategic and International Studies. *Earth Orbit 101*. Accessed: 2024-12-09. 2024. URL: <https://aerospace.csis.org/aerospace101/earth-orbit-101/>.
- [5] N2YO. *International Space Station (ISS) - Satellite Tracker*. Accessed: 2024-12-10. 2024. URL: <https://www.n2yo.com/satellite/?s=25544#results>.
- [6] Starlink. *Starlink Technology*. Accessed: 2024-12-10. 2024. URL: <https://www.starlink.com/us/technology>.
- [7] TechTarget. *Geostationary Satellite Definition*. Accessed: 2024-12-10. 2024. URL: <https://www.techtarget.com/searchmobilecomputing/definition/geostationary-satellite#:~:text=A%20geostationary%20satellite%20is%20an,kilometers%20directly%20above%20the%20equator>.
- [8] The Economist. "Attacking satellites is increasingly attractive—and dangerous". In: *The Economist* (2019). Accessed: 2024-12-11. URL: <https://www.economist.com/briefing/2019/07/18/attacking-satellites-is-increasingly-attractive-and-dangerous>.
- [9] Max Roser. *A Record Number of Objects Went into Space in 2023*. Accessed: December 9, 2024. 2023. URL: <https://ourworldindata.org/data-insights/a-record-number-of-objects-went-into-space-in-2023>.
- [10] Space-Track.org. *Space-Track.org*. Accessed: 2024-12-10. 2024. URL: <https://www.space-track.org/>.
- [11] Celestrak. *TLE Format Documentation*. Accessed: 2024-12-10. 2024. URL: <https://celestrak.org/NORAD/documentation/tle-fmt.php>.
- [12] Rhodes Mill. *Skyfield Documentation*. Accessed: 2024-12-10. 2024. URL: <https://rhodesmill.org/skyfield/toc.html>.
- [13] National Weather Service. *Twilight Types*. Accessed: 2024-12-10. 2024. URL: <https://www.weather.gov/lmk/twilight-types#:~:text=Astronomical%20Twilight%3A,urban%20or%20suburban%20light%20pollution>.
- [14] V. Gupta. "Reinforcement Learning Explained Visually: Part 4 - Q-learning Step-by-Step". In: *Towards Data Science* (2020). Accessed: 2024-12-13. URL: <https://towardsdatascience.com/reinforcement-learning-explained-visually-part-4-q-learning-step-by-step-b65efb731d3e#:~:text=The%20Q%2Dlearning%20algorithm%20uses,the%20Q%2Dvalues%20to%20zero..>
- [15] Daniel Takeshi. *Going Deeper into Reinforcement Learning: Understanding Q-learning and Linear Function Approximation*. Accessed: 2024-12-13. 2016. URL: <https://danieltakeshi.github.io/2016/10/31/going-deeper-into-reinforcement-learning-understanding-q-learning-and-linear-function-approximation/>.